

중간고사

강화학습 중간고사 상세 해설집

📖 Ch.01 Introduction (강화학습 개요)

🔍 핵심 개념 상세

1) 머신러닝 3분류

****머신러닝(Machine Learning)**** 은 "데이터로부터 패턴을 학습하는 알고리즘의 총칭"으로, 데이터의 ****레이블 유무****와 ****학습 신호의 종류****에 따라 3가지로 구분됩니다.

- ****지도학습 (Supervised Learning)****

- 입력 x 와 정답 레이블 y 가 쌍으로 주어진 데이터 (x, y) 에서 함수 $f: x \rightarrow y$ 를 학습
- 학습 신호: ****정답(label)****
- 예: 회귀(집값 예측), 분류(스팸 메일, 객체 탐지)
- 알고리즘: 선형회귀, 결정트리, 신경망

- ****비지도학습 (Unsupervised Learning)****

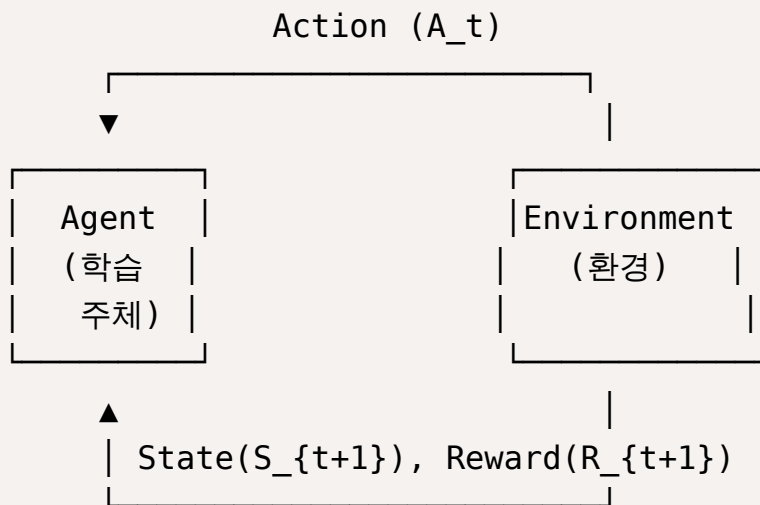
- 정답 없이 입력 x 만으로 데이터의 구조/패턴 발견
- 학습 신호: ****데이터 자체의 구조****
- 예: 클러스터링(K-means), 차원축소(PCA)

- ****강화학습 (Reinforcement Learning)****

- 환경과 상호작용하며 ****보상(reward)**** 으로 학습
- 학습 신호: ****보상(스칼라 값)****
- 데이터가 미리 주어지지 않고, 시행착오(trial-and-error)를 통해 직접 수집
- 예: 게임 AI, 자율주행, 로봇 제어

2) 강화학습의 5요소

...



...

요소	정의	로봇 보행 예시
Agent (학습 주체)	의사결정자, 행동을 선택	로봇 제어기
Environment (환경)	Agent와 상호작용하는 모든 것	도로, 장애물, 중력
State (상태)	환경의 현재 상황을 나타내는 정보	장애물 위치, 관절 각도
Action (행동)	Agent가 선택할 수 있는 동작	다리 관절의 회전 각도/속도
Reward (보상)	행동의 결과로 받는 스칼라 평가값	이동 거리, 넘어지지 않은 시간

****목표****: 누적 보상의 기댓값(expected return)을 최대화하는 정책(policy)을 학습.

📌 예상 문제 정답 풀이

[Q1-1] 지도/비지도/강화학습 비교

****모범답안:****

구분	학습 데이터	학습 방식
----	--------	-------

```
|-----|-----|-----|
| 지도학습 | 입력 + 정답 (labeled) | 정답과 예측의 오차 최소화 |
| 비지도학습 | 입력만 (unlabeled) | 데이터의 분포/구조 학습 |
| 강화학습 | 환경에서 직접 수집한 (s, a, r, s') | 보상의 기댓값 최대화 |
```

****핵심 차이****: 지도학습은 "정답"을 따라하지만, 강화학습은 ****순차적 의사결정**** 문제이며 한 행동이 미래의 모든 보상에 영향을 미친다는 점에서 본질적으로 다릅니다.

****[Q1-2]**** 로봇 보행 문제의 5요소 매핑

****모범답안:****

- ****Agent****: 로봇 제어기 (control policy)
- ****Environment****: 도로, 장애물, 중력 등 외부 세계
- ****State****: 장애물의 위치, 로봇의 관절 각도/속도, 자세
- ****Action****: 각 다리 관절의 이동 각도 또는 토크
- ****Reward****: 이동한 거리(+), 넘어짐(-), 에너지 소모(-)

****[Q1-3]**** RL과 supervised learning의 본질적 차이

****모범답안:****

1. ****학습 신호****: 지도학습은 정답(label)을 따라하지만, 강화학습은 평가성 신호인 ****보상(reward)**** 만 받음. 어떤 행동이 "정답"인지는 직접 알려주지 않음.
2. ****순차성****: 강화학습은 ****순차적 의사결정 문제(sequential decision-making)**** 로, 현재 행동이 미래 상태/보상에 영향을 줌. 지도학습은 i.i.d. 가정 하에 각 샘플을 독립적으로 다룸.
3. ****데이터 분포의 영향****: 강화학습은 정책에 따라 데이터 분포가 바뀜 (non-stationary). 지도학습은 고정된 분포를 가정.

 Ch.02 Bandit Problem

🔍 핵심 개념 상세

1) Multi-armed Bandit 문제

****Bandit**** = 슬롯머신. ****Multi-armed bandit****은 N개의 슬롯머신이 각각 다른 승률을 가질 때, 정해진 횟수 안에 최대 코인을 얻는 문제입니다.

- 처음에는 각 머신의 승률을 모름
- 플레이를 통해 정보를 수집하면서 좋은 머신을 찾아야 함
- ****State는 정의되지 않음**** (가장 단순한 형태의 RL 문제)

2) Action Value (행동 가치)

각 행동 a에 대한 ****보상의 기댓값****:

...

$$Q(a) = E[R \mid A = a]$$

...

예: 슬롯머신 a의 승률이 0.6, 보상 = 1코인이면 $Q(a) = 0.6$.

> 💡 ****쉽게 이해하기****

>

> - ****읽는 법:**** "행동 a를 골랐을 때 받게 될 보상 R의 평균(기댓값)이 $Q(a)$ 다."

> - ****기호 풀이:****

> - R = 이번 판에 받은 보상 (예: 0 또는 1코인)

> - $A = a$ = "행동 a를 선택했다는 조건"

> - $E[\dots \mid \dots]$ = "조건이 주어졌을 때의 평균"

> - ****직관:**** 슬롯머신 a를 ****무한히 많이 당겼을 때****의 평균 수익.

> - ****예시 계산:**** 승률 0.6, 이기면 1코인 / 지면 0코인 $\rightarrow Q(a) = 0.6 \times 1 + 0.4 \times 0 = 0.6$ 코인.

> - ****한 줄로:**** $Q(a)$ 는 "그 팔(action)의 장기 평균 성능 점수".

3) Sample Average와 Incremental Update

n번의 플레이 후 평균 보상:

...

$$Q_n = (R_1 + R_2 + \dots + R_n) / n$$

...

> 💡 **쉽게 이해하기 (표본평균)**

>

> - **읽는 법:** "지금까지 받은 모든 보상을 더해 횟수로 나눈 것 = 평균."

> - **왜 쓰나:** 진짜 $Q(a)$ 는 모르므로, **관측한 데이터로 추정**한다. 많이 당길수록 진짜 평균에 가까워진다(대수의 법칙).

> - **예:** 보상이 [1, 0, 5]면 $Q_3 = (1+0+5)/3 = 2$.

****점진적(Incremental) 갱신식**** (메모리 효율을 위해 모든 R을 저장하지 않음):

...

$$Q_n = Q_{n-1} + (1/n) \cdot (R_n - Q_{n-1})$$

...

****유도:****

...

$$\begin{aligned} Q_n &= (1/n) \cdot \sum_{i=1..n} R_i \\ &= (1/n) \cdot [R_n + \sum_{i=1..n-1} R_i] \\ &= (1/n) \cdot [R_n + (n-1) \cdot Q_{n-1}] \\ &= R_n / n + Q_{n-1} - Q_{n-1} / n \\ &= Q_{n-1} + (1/n) \cdot (R_n - Q_{n-1}) \end{aligned}$$

...

여기서 $(1/n)$ 은 ****학습률(learning rate)**** 역할.

> 💡 **쉽게 이해하기 (점진식)**

>

> - **읽는 법:** "새 평균 = 옛 평균 + $(1/n) \times (\text{이번 보상} - \text{옛 평균})$ "

> - **모양의 의미:** **새 추정 = 옛 추정 + 학습률 $\times$ (관측 - 옛 추

정)** - 강화학습 모든 갱신식의 **기본 패턴**.

> - **$(R_n - Q_{n-1})$ 의 뜻**: "방금 받은 보상이 지금까지 평균보다 얼마나 큰가" = **오차/잔차**.

> - **$(1/n)$ 의 뜻**: 그 오차를 **얼마나 반영할지의 비율**. n 이 커질수록 새 보상 하나의 영향이 작아짐(평균이 점점 안정).

> - **장점**: 과거 보상 $R_1, R_2, \dots$ 를 **저장할 필요 없음**. 옛 평균 Q_{n-1} 과 횟수 n , 새 보상 R_n 만 있으면 됨.

> - **단계별 예 (보상 $[1, 0, 5]$, $Q_0 = 0$)**:

> - $Q_1 = 0 + (1/1)(1-0) = 1$

> - $Q_2 = 1 + (1/2)(0-1) = 0.5$

> - $Q_3 = 0.5 + (1/3)(5-0.5) = 2.0$ ← 표본평균 $(1+0+5)/3 = 2$ 와 동일

4) Exploitation vs Exploration

- **Exploitation (활용)**: 지금까지 학습한 Q 를 토대로 가장 좋은 행동 선택 (greedy)
- **Exploration (탐색)**: 더 나은 행동을 발견하기 위해 무작위 행동 시도

ϵ -greedy 정책:

- 확률 ϵ 로 random (exploration)
- 확률 $(1-\epsilon)$ 로 greedy (exploitation)
- $\epsilon = 0 \rightarrow$ 완전 greedy (exploitation only)
- $\epsilon = 1 \rightarrow$ 완전 random (exploration only)
- 일반적으로 $\epsilon \approx 0.1$ 정도 사용

5) Stationary vs Non-stationary

- **Stationary**: 슬롯머신의 승률이 고정. 표본평균($1/n$) 사용 적절.
- **Non-stationary**: 승률이 시간에 따라 변함. 최신 데이터에 더 큰 가중치 필요.

고정 step size α 사용:

...

$$Q_n = Q_{n-1} + \alpha \cdot (R_n - Q_{n-1})$$

...

****전개:****

...

$$\begin{aligned} Q_n &= (1-\alpha) \cdot Q_{n-1} + \alpha \cdot R_n \\ &= (1-\alpha)^n \cdot Q_0 + \sum_{i=1..n} \alpha \cdot (1-\alpha)^{n-i} \cdot R_i \end{aligned}$$

...

가중치 $\alpha \cdot (1-\alpha)^{n-i}$ 는 i 가 작을수록(과거일수록) 지수적으로 감소 → 최신 보상에 더 큰 비중.

> 💡 ****쉽게 이해하기 (고정 α)****

>

> - ****(1/n) vs α 차이:****

> - ****`(1/n)`****: n 이 커질수록 작아짐 → 모든 보상을 동등 가중. ***stationary***(분포 안 바뀜)에 적합.

> - ****` $\alpha$ ` (예: 0.1)****: 항상 같은 비율 → ****최신 보상에 더 큰 비중****. ****non-stationary****(분포가 바뀜)에 적합.

> - ****두 번째 줄 전개의 뜻:**** 새로운 Q_n 을 풀어 쓰면, 과거 보상 R_i 가 ****(1- α)⁽ⁿ⁻ⁱ⁾**** 만큼 곱해져 가중합된다.

> - $n-i$ 가 클수록 (= 옛날 보상일수록) **`(1- $\alpha$ )`**를 여러 번 곱하니까 ****점점 작아진다(지수 감소)****.

> - 결과: ****최근 데이터는 크게 반영, 오래된 데이터는 거의 무시****.

> - ****비유:**** 시험 점수 평균을 낼 때, ****최근 시험에 더 가중치****를 둔다고 생각하면 됨.

> - ****언제 쓰나:**** 환경이 시간에 따라 변할 때(승률이 바뀌는 슬롯), 또는 학습 초기의 부정확한 추정을 빨리 잊고 싶을 때.

📌 예상 문제 정답 풀이

**[Q2-1]**** Action value 정의 + incremental 식 유도**

****정의:****

...

```
Q(a) = E[ R | A = a ]
...
```

행동 a를 선택했을 때 받는 보상의 기댓값.

****표본평균 추정:****

```
...
```

```
Q_n = (1/n) · Σ_{i=1..n} R_i
...
```

****Incremental 유도**** (위 4) 참고):

```
...
```

```
Q_n = Q_{n-1} + (1/n) · ( R_n - Q_{n-1} )
...
```

****의미****: "현재 추정치 + 학습률 × (새 관측값 - 현재 추정치)" 형태로, 모든 강화학습 갱신식의 기본 패턴입니다.

```
---
```

**[Q2-2] ε-greedy 정책**

****설명****: 확률 ε로 무작위 행동(exploration), (1-ε)로 가장 큰 Q값을 가진 행동(exploitation)을 선택하는 정책.

- ****ε = 0****: 완전한 greedy. 한 번 좋다고 판단된 행동만 계속 선택 → exploration 없음 → 더 좋은 행동을 놓칠 위험.
- ****ε = 1****: 완전한 random. 학습한 정보를 전혀 활용 안 함 → 비효율.

****실용적 선택****: 초반에는 ε를 크게(탐색), 점차 작게(활용) 줄이는 ****decaying ε-greedy****가 일반적.

```
---
```

**[Q2-3] Stationary vs Non-stationary**

항목	Stationary	Non-stationary
환경	보상 분포 고정	보상 분포 변화
가중치	모든 보상에 동일($1/n$)	최신 보상에 더 큰 가중치(α)
갱신식	$Q_n = Q_{n-1} + (1/n)(R_n - Q_{n-1})$	$Q_n = Q_{n-1} + \alpha(R_n - Q_{n-1})$

****고정 α 사용 이유****: 비정상 환경에서는 과거 데이터가 현재와 무관할 수 있음. $1/n$ 은 n 이 커질수록 새 데이터의 영향이 작아져 변화 추종 불가. α =고정값이면 가중치가 지속적으로 감소하여 항상 최신 데이터를 잘 반영.

****[Q2-4]**** 수치 계산 (보상 [1, 0, 5], $Q_0 = 0$)

단계	계산	결과
Q_1	$0 + (1/1)(1 - 0)$	**1**
Q_2	$1 + (1/2)(0 - 1) = 1 - 0.5$	**0.5**
Q_3	$0.5 + (1/3)(5 - 0.5) = 0.5 + 1.5$	**2.0**

검증: $(1 + 0 + 5) / 3 = 2.0 \checkmark$

📖 Ch.03 Markov Decision Process (MDP) ★★★

🔍 핵심 개념 상세

1) Markov Property

****정의:****

...

$\Pr(S_{t+1} \mid S_0, S_1, \dots, S_t) = \Pr(S_{t+1} \mid S_t)$

...

****의미****: 현재 상태가 주어지면, 미래는 과거와 독립적. 즉, 현재 상태 S_t 가 과거 모든 정보를 충분히 요약(sufficient statistic)함.

> "The future is independent of the past given the present."

> 💡 ****쉽게 이해하기 (마르코프 성질)****

>

> - ****시간 첨자 정리****

> - ' $S_0, S_1, \dots, S_{t-1}$ ' = ****과거**** 상태들

> - ' S_t ' = ****지금(현재)**** 상태

> - ' S_{t+1} ' = ****다음**** 상태(바로 한 칸 앞 미래)

> - ****읽는 법****: "다음 상태 ' S_{t+1} '의 확률을 예측할 때, 과거 전체(' $S_0 \dots S_{t-1}$ ')를 알 필요 없이 ****현재 ' S_t '만 알면 충분****하다."

> - ****비유****: 체스에서 ****현재 말 위치****만 보면 다음 수가 결정된다

→ 어떤 순서로 그 위치까지 왔는지는 몰라도 됨.

> - ****반례(마르코프 아님)****: "어제 비가 왔는지"를 모르면 다음 날씨를 못 맞추는 경우 → 이런 정보는 ****상태에 포함****시켜야 마르코프가 됨.

****RL에서의 표현****:

...

$$\Pr(R_{t+1} = r, S_{t+1} = s' \mid S_0, A_0, R_1, \dots, S_t, A_t)$$

$$= \Pr(R_{t+1} = r, S_{t+1} = s' \mid S_t, A_t)$$

$$= p(s', r \mid s, a)$$

...

> 💡 ****쉽게 이해하기 (RL 버전)****

>

> - ****읽는 법****: "다음 보상 ' R_{t+1} '과 다음 상태 ' S_{t+1} '이 어떻게 나올지는, ****지금 상태 ' S_t '와 방금 한 행동 ' A_t '만 알면**** 결정된다."

> - ****세 줄이 같은 이유****

> - 첫 줄: 과거 전체(' $S_0, A_0, R_1, \dots, S_t, A_t$ ')를 조건으로 둬.

```
> - 둘째 줄: 마르코프라서 과거를 버리고 `(S_t, A_t)`만 남김.
> - 셋째 줄: `S_t = s, A_t = a` 로 변수 이름을 짧게 바꿔 **함수 이름 `p(s', r | s, a)`** 로 부르기로 약속.
> - **`p(s', r | s, a)`의 뜻:** "s에서 a를 하면, **다음에 s'로 가면서 보상 r을 받을 확률**" - 한 스텝의 모든 정보를 담은 **원자(atom)**.
```

2) MDP의 정의 (5튜플)

```
...
```

```
MDP = (  $\mathcal{S}$ ,  $\mathcal{A}$ , T, r,  $\gamma$  )
...
```

- $\mathcal{S}$: 상태 집합
- $\mathcal{A}$: 행동 집합
- $T = p(s' | s, a)$: 상태 전이 확률
- $r(s, a, s')$: 보상 함수
- γ : 할인율 ($0 \leq \gamma \leq 1$)

```
> 💡 **쉽게 이해하기 (MDP 5튜플)**
```

```
>
```

```
> - **읽는 법:** "MDP는 환경을 수학으로 적기 위한 **재료 5가지 묶음**이다."
```

```
> - **각 재료의 역할:**
```

```
> - ** $\mathcal{S}$  (상태 집합):** 에이전트가 있을 수 있는 모든 위치/상황.
예: 그리드월드의 모든 칸.
```

```
> - ** $\mathcal{A}$  (행동 집합):** 고를 수 있는 모든 행동. 예: {위, 아래,
왼쪽, 오른쪽}.
```

```
> - **T (전이 확률):** 현재 (s, a)에서 다음에 어디로 갈지의 확률 규칙.
```

```
> - **r (보상 함수):** 한 스텝마다 환경이 주는 점수 규칙.
```

```
> - ** $\gamma$  (할인율):** **미래 보상을 현재 가치로 얼마나 줄여서** 평가할지(0~1 사이). 가까운 보상을 더 중요하게.
```

```
> - **비유:** 게임을 만든다고 생각 - 맵( $\mathcal{S}$ ), 조작( $\mathcal{A}$ ), 이동 규칙(T), 점수 규칙(r), 미래 점수 가중치( $\gamma$ ).
```

```
**One-step dynamics:**
```

...

$$p(s', r | s, a) = \Pr(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$$

...

> 💡 ****쉽게 이해하기 (one-step dynamics)****

>

> - ****읽는 법:**** "한 스텝의 환경 동작을 ****확률표 한 장****으로 적은 것."

> - ****의미:**** s에서 a를 하면 ****(다음 상태 s', 보상 r)**** 짝이 나오는데, 그 짝의 확률.

> - ****모든 (s', r) 조합을 다 더하면 1****: $\sum_{s', r} p(s', r | s, a) = 1$

> - ****중요한 이유:**** 아래 두 수식(전이확률, 기대보상)은 모두 이 한 식에서 ****합/평균으로 뽑아낸 것****.

****상태 전이 확률:****

...

$$p(s' | s, a) = \sum_r p(s', r | s, a)$$

...

> 💡 ****쉽게 이해하기 (상태 전이 확률)****

>

> - ****읽는 법:**** "보상이 무엇이든 신경 쓰지 말고, ****다음 상태가 s'일 확률만**** 구하자."

> - ****하는 일:**** $p(s', r | s, a)$ 표에서, 같은 s'를 가지는 행들의 확률을 ****r에 대해 다 더함**** (= 주변화, marginalization).

> - ****예시:**** s=비움, a=우산쓰일 때 표가 다음과 같다고 하자:

> | (s', r) | 확률 |

> | --- | --- |

> | (집도착, +1) | 0.5 |

> | (집도착, -1) | 0.3 |

> | (젖음, -2) | 0.2 |

> - $p(\text{집도착} | s, a) = 0.5 + 0.3 = 0.8$

> - $p(\text{젖음} | s, a) = 0.2$

****기대 보상:****

...

$$r(s, a) = E[R_{t+1} \mid S_t = s, A_t = a] \\ = \sum_{s', r} r \cdot p(s', r \mid s, a)$$

$$r(s, a, s') = E[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s']$$

...

> 💡 **쉽게 이해하기 (기대 보상)**

>

> - **`r(s, a)`**: "s에서 a를 했을 때 받을 보상의 **평균**" (다음 상태가 어디든 상관없이 전부 평균).

> - 계산법: 표에서 **각 칸마다 (보상값 × 그 칸의 확률)** 을 더한다.

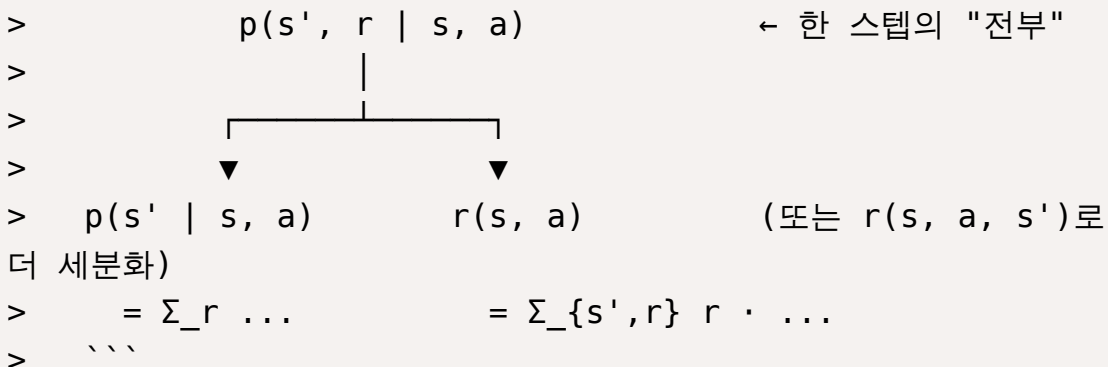
> - 위 예: $r(s, a) = 1 \cdot 0.5 + (-1) \cdot 0.3 + (-2) \cdot 0.2 = 0.5 - 0.3 - 0.4 = -0.2$

> - **`r(s, a, s')`**: "다음 상태가 **s'로 정해졌다고 알려준 뒤**의 보상 평균".

> - 위 예에서 s'=집도착으로 조건을 걸면, 가능한 r은 +1(0.5), -1(0.3) → 조건부 평균 $(1 \cdot 0.5 + (-1) \cdot 0.3) / (0.5 + 0.3) = 0.25$.

> - **요약 그림**:

> ...



3) Episodic vs Continuing Task

- **Episodic task**: 종결 상태(terminal state)에서 끝남. 예: 바둑, 게임, Grid World.

- **Continuing task**: 끝이 없음. 예: 재고관리, Recycling Robot.

4) Return (수익)

****정의****: 시점 t 이후 받게 될 보상의 합.

- ****Episodic**** ($T = \text{종결 시점}$):

...

$$G_t = R_{\{t+1\}} + R_{\{t+2\}} + \dots + R_T$$

...

- ****Continuing**** ($T = \infty$):

...

$$G_t = R_{\{t+1\}} + \gamma \cdot R_{\{t+2\}} + \gamma^2 \cdot R_{\{t+3\}} + \dots$$
$$= \sum_{k=0..\infty} \gamma^k \cdot R_{\{t+k+1\}}$$

...

****할인율 γ 의 역할****

- $\gamma = 0$: 즉각적 보상만 고려 (myopic)
- $\gamma \rightarrow 1$: 먼 미래 보상도 동등하게 중요시
- $\gamma = 1 + \text{continuing task} \rightarrow G_t$ 가 발산 (∞)
- 일반적으로 0.9 ~ 0.99 사용

> 💡 ****쉽게 이해하기 (Return G_t)****

>

> - ****읽는 법**** "지금 시점 t 이후로 ****앞으로 받을 보상을 다 합친 값****."

> - ****첨자 주의**** ' G_t '는 t ****다음****($t+1$)부터의 보상을 모은 것 (' $R_{\{t+1\}}$ '부터 시작).

> - ****왜 γ 를 곱하나****

> - ****수렴 보장**** 끝없이 더하면 무한이 될 수 있음. $\gamma < 1$ 이면 등비수열로 유한 값에 수렴.

> - ****시간 선호**** 똑같은 1만원도 ****오늘이 내일보다 가치 있다****는 사람의 직관 반영.

> - ****불확실성**** 먼 미래는 무슨 일이 생길지 모르니 가치를 깎음.

> - ****숫자 예 ($\gamma = 0.9$, 보상 $[10, 5, 2, 1]$):****

$$G_0 = 10 + 0.9 \cdot 5 + 0.9^2 \cdot 2 + 0.9^3 \cdot 1 = 10 + 4.5 + 1.62 + 0.729 = 16.85$$

```
> - 미래로 갈수록 0.9, 0.81, 0.729, ... 처럼 **점점 작게 반영*
*됨.
> - **재귀 형태(아래 Bellman의 핵심 트릭):** `G_t = R_{t+1} + \gamma
\cdot G_{t+1}` - "지금 한 스텝 보상 + \gamma \times 다음 시점부터의 G".
```

5) Value Function

****State-value function**** (정책 π 하에서 상태 s 의 가치):

```
...
```

```
v_\pi(s) = E_\pi[ G_t | S_t = s ]
```

```
...
```

"상태 s 에서 출발해 정책 π 를 따랐을 때의 기대 누적 보상."

```
> 💡 **쉽게 이해하기 (v_\pi)**
```

```
>
```

```
> - **읽는 법:** "정책  $\pi$ 를 쓰는 사람이 **상태  $s$ 에 서 있을 때**,
앞으로 받을 보상의 평균이  $v_\pi(s)$ 다."
```

```
> - **`E_\pi[ ... ]`의 뜻:** "정책  $\pi$ 에 따라 행동·전이가 일어날 때의
기대값."
```

```
> - **비유:** "이 자리에서 이 전략을 쓰면 **평균적으로 점수 얼마 받
겠다**" 라는 평가표.
```

****Action-value function**** (Q-function):

```
...
```

```
q_\pi(s, a) = E_\pi[ G_t | S_t = s, A_t = a ]
```

```
...
```

"상태 s 에서 행동 a 를 한 후, 이후 정책 π 를 따랐을 때의 기대 누적 보
상."

```
> 💡 **쉽게 이해하기 (q_\pi)**
```

```
>
```

```
> - **읽는 법:** "상태  $s$ 에서 **딱 이번에는 행동  $a$ 를 강제로** 하고,
**그 다음부터는 정책  $\pi$ 를 따를 때**의 평균 누적 보상."
```

```
> - **v_\pi와의 차이:** v_\pi는 "그 자리의 가치"이고, q_\pi는 "그 자리
```

에서 ****특정 행동을 했을 때****의 가치".

> - ****왜 둘 다 필요한가:**** 정책을 ****개선****하려면 "어느 행동이 더 나은가?"를 비교해야 하는데, 그건 q_π 로 직접 알 수 있다 (Ch.5의 "Q를 추정해야 하는 이유").

****State-value와 Action-value의 관계:****

...

$$v_\pi(s) = \sum_a \pi(a | s) \cdot q_\pi(s, a)$$

$$q_\pi(s, a) = \sum_{s'} p(s' | s, a) \cdot [r(s, a, s') + \gamma \cdot v_\pi(s')]$$

...

> 💡 ****쉽게 이해하기 ($v \leftrightarrow q$ 관계)****

>

> - ****첫 식:**** " $v_\pi(s)$ 는 $q_\pi(s, a)$ 들의 ****정책 가중 평균****."

> - $\pi(a | s)$: 정책이 상태 s 에서 행동 a 를 고를 확률.

> - 모든 행동 a 에 대해 (그 행동을 고를 확률 $\times$ 그 행동의 가치)를 더하면 $\rightarrow$ 그 자리의 평균 가치.

> - ****예:**** π 가 [위 0.5, 아래 0.5]이고 q 값이 [위 4, 아래 2]면, $v = 0.5 \cdot 4 + 0.5 \cdot 2 = 3$.

> - ****둘째 식:**** " $q_\pi(s, a)$ 는 ****다음 상태들의 가치를 평균****한 것."

> - $p(s' | s, a)$: 그 행동을 했을 때 s' 로 갈 확률.

> - $r(s, a, s')$: 거기서 갔을 때 받는 보상.

> - $\gamma \cdot v_\pi(s')$: 다음 자리에서의 미래 가치(할인됨).

> - ****즉:**** "이번에 받을 보상 $+ \gamma \times$ 다음 자리의 가치"를 가능한 모든 s' 에 대해 평균.

> - ****두 식을 합쳐 v 를 q 로 적으면 $\rightarrow$ Bellman 방정식.**** (다음 절에서 유도)

6) Bellman Equation 유도 (가장 중요!)

****Step 1:**** Return의 재귀 관계

...

$$G_t = R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots$$


```

    = R_{t+1} + \gamma \cdot ( R_{t+2} + \gamma \cdot R_{t+3} + \dots )
    = R_{t+1} + \gamma \cdot G_{t+1}
    ...

> 💡 **쉽게 이해하기 (Step 1)**
>
> - **읽는 법:** "G_t = (이번 보상) + \gamma \times (다음부터의 G)"
> - **두 줄 변환의 트릭:** \gamma를 두 번째 항부터 묶어내면, 안쪽 괄호가
    정확히 G_{t+1}의 정의가 됨.
> - **왜 중요한가:** 무한합처럼 보이던 G_t가 **두 부분**(즉시 보상
    + 미래 G의 할인)으로 깔끔히 갈라짐 → Bellman의 핵심 트릭.

```

****Step 2**:** 정의에 대입

```

    ...
v_\pi(s) = E_\pi[ G_t | S_t = s ]
           = E_\pi[ R_{t+1} + \gamma \cdot G_{t+1} | S_t = s ]
           = E_\pi[ R_{t+1} | S_t = s ] + \gamma \cdot E_\pi[ G_{t+1} | S_t
           = s ]
                                           (← 기댓값의 선형성)
    ...

```

```

> 💡 **쉽게 이해하기 (Step 2)**
>
> - **하는 일:** v_\pi 정의에 Step 1 결과를 넣고, 기댓값을 두 부분
    으로 나눔.
> - **기댓값 선형성:** `E[X + Y] = E[X] + E[Y]` (확률에 상관없
    이 항상 성립). \gamma는 상수라 밖으로 빼짐.
> - **결과 해석:** "v_\pi(s) = (즉시 보상의 기대) + \gamma \times (다음부터
    의 G의 기대)" - 첫 항은 한 스텝 보상, 둘째 항은 다음 자리의 가치.

```

****Step 3**:** 각 항 전개

- 첫째 항:
...

```

E_\pi[ R_{t+1} | S_t = s ] = \sum_{a, s'} \pi(a|s) \cdot p(s'|s, a) \cdot
r(s, a, s')
    ...

```

- 둘째 항:

...

$$E_{\pi}[G_{t+1} \mid S_t = s] = \sum_{a, s'} \pi(a|s) \cdot p(s'|s, a) \cdot v_{\pi}(s')$$

...

> 💡 **쉽게 이해하기 (Step 3)**

>

> - **두 종류 우연을 풀어 적기:**

> - 행동 a 는 **정책 π ** 가 확률적으로 고름 → 가중치 ' $\pi(a|s)$ '

> - 다음 상태 s' 은 **환경 p ** 가 확률적으로 결정 → 가중치 ' $p(s'|s, a)$ '

> - 두 확률을 곱한 뒤, 가능한 (a, s') 모든 조합에 대해 합 = **기댓값 정의**.

> - **첫째 항:** 그 (a, s') 조합에서 받는 보상의 평균 → $r(s, a, s')$ 에 가중치 곱.

> - **둘째 항:** "다음 자리에서 앞으로 받을 G 의 기대" = **다음 자리의 $v_{\pi}(s')$ ** (정의상).

> - **이 두 항을 묶으면 → 최종 Bellman equation** (다음 박스).

최종 Bellman Equation (state-value):

...

$$v_{\pi}(s) = \sum_{a, s'} \pi(a|s) \cdot p(s'|s, a) \cdot [r(s, a, s') + \gamma \cdot v_{\pi}(s')]$$

...

> 💡 **쉽게 이해하기 (Bellman equation, 상태가치)**

>

> - **한 줄 의미:** "***현재 자리의 가치 = 다음 한 스텝에 일어날 모든 (a, s') 조합을 평균낸 값**."

> - **각 부분 해석:**

> - ' $\pi(a|s)$ ': "***정책 π 가** 이 상태에서 a 를 고를 확률" (행동

평균)

```
> - `p(s'|s, a)` : "그 행동 결과 s'로 갈 확률" (전이 평균)
> - `r(s, a, s')` : "이번에 받을 보상"
> - `γ · v_π(s')` : "다음 자리에서 앞으로 받을 가치(할인됨)"
> - **Σ 두 개의 의미:**
>   - 가능한 모든 행동 a (여러 개 선택지)
>   - 그 행동에 대한 가능한 모든 다음 상태 s' (여러 결과)
>   - 두 종류 우연(행동 + 환경)을 **각각의 확률로 가중평균**한 것.
> - **재귀(자기 참조):** 우변에 또 `v_π(s')`이 등장. → 모든 상태
  의 가치가 **서로 의존**하는 **연립방정식**.
> - **숫자 그림:** "이 칸의 점수 = (각 행동 확률 × 그 행동을 했을
  때 평균 결과)의 합".
```

****Bellman Equation (action-value):****

...

$$q_{\pi}(s, a) = \sum_{s'} p(s'|s, a) \cdot [r(s, a, s') + \gamma \cdot \sum_{a'} \pi(a'|s') \cdot q_{\pi}(s', a')]$$

...

> 💡 ****쉽게 이해하기 (Bellman equation, 행동가치)****

>

> - ****읽는 법:**** "이 자리에서 ****이 행동을 한 가치 = 그 결과로 갈 다음 자리들의 q를 평균낸 값****."

> - ****유도 트릭:**** $v_{\pi}(s') = \sum_{a'} \pi(a'|s') \cdot q_{\pi}(s', a')$

이므로, 첫 식의 v 대신 q들의 평균으로 치환한 것.

> - ****차이점:**** 행동가치 식은 ****q끼리 재귀****한다 → q를 직접 학습하면 모델 없이 정책 비교 가능.

7) Optimal Value Function & Policy

****Optimal state-value:****

...

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

...

****Optimal action-value:****

```

'''
q*(s, a) = max_π q_π(s, a)
'''

**Optimal action (deterministic):**

'''
μ*(s) = argmax_a q*(s, a)
'''

**Optimal policy:**

'''
π*(a|s) = 1    if a = μ*(s)
          = 0    otherwise
'''

### 8) Bellman Optimality Equation

**State-value:**

'''
v*(s) = max_a Σ_{s'} p(s'|s, a) · [ r(s, a, s') + γ · v*
(s') ]
'''

**Action-value:**

'''
q*(s, a) = Σ_{s'} p(s'|s, a) · [ r(s, a, s') + γ · max_
{a'} q*(s', a') ]
'''

> **Bellman equation = 연립방정식 문제**
> **Bellman optimality equation = 최적화 문제** (max 때문에 비
선형)

```

```

> 💡 **쉽게 이해하기 (Bellman optimality)**
>
> - **일반 Bellman과의 차이:**  $\sum_a \pi(a|s) \cdot \dots$  (행동 확률로
평균)  $\rightarrow \max_a \dots$  (최선의 행동만 선택)
> - **읽는 법:**
>   -  $v^*(s)$ : "이 자리에서 **항상 최선의 행동만 골랐을 때**의
가치."
>   -  $q^*(s, a)$ : "이번에 a를 한 뒤, **그 다음부터는 매번 최선
의 행동**을 골랐을 때의 가치."
> - **정책에서 출발하지 않는 이유:** 최적 정책이 무엇인지 모를 때
도, **각 자리에서 그저 최선만 고른다**는 가정으로 만든 식.
> - **풀이가 어려운 이유:**  $\max$ 는 비선형  $\rightarrow$  일반 연립방정식처럼
한 번에 못 풀고, **반복 계산(Value Iteration)** 으로 풀어야 함.
> - **Optimal policy 추출:**  $v^*$ 나  $q^*$ 를 알면,  $\pi^*(s) = \operatorname{argmax}_a q^*(s, a)$ 
로 한 번에 최적 정책이 나옴.
> - **요약 비교:**
>   | | 일반 Bellman (정책  $\pi$ ) | Bellman Optimality |
>   |---|---|---|
>   | 행동 처리 |  $\pi(a|s)$ 로 가중평균 |  $\max_a$ 로 최선 |
>   | 의미 | "이 정책의 가치" | "최적 정책의 가치" |
>   | 푸는 도구 | Policy Evaluation | Value Iteration |

```

➡ 예상 문제 정답 풀이

**[Q3-1]**** Markov Property**

****수식:****

...

$\Pr(S_{\{t+1\}} \mid S_0, S_1, \dots, S_t) = \Pr(S_{\{t+1\}} \mid S_t)$

...

****의미****: 현재 상태가 미래에 대한 모든 정보를 충분히 담고 있어, 과거 정보는 추가로 필요 없음. $\rightarrow$ "Future $\perp$ Past | Present"

[Q3-2] Return 정의 + γ 필요 이유

- **Episodic**: $G_t = R_{\{t+1\}} + R_{\{t+2\}} + \dots + R_T$
- **Continuing**: $G_t = \sum_{k=0..\infty} \gamma^k \cdot R_{\{t+k+1\}}$

γ 가 필요한 이유:

1. **수렴성 보장**: continuing task에서 $\gamma = 1$ 이면 무한 합이 발생. $\gamma < 1$ 이어야 $|R| \leq R_{\max}$ 인 한 G_t 는 $R_{\max} / (1-\gamma)$ 로 유한.
2. **시간 선호 반영**: 가까운 보상이 먼 보상보다 더 중요하다는 사람의 선호와 부합.
3. **불확실성 반영**: 먼 미래는 불확실성이 크므로 가치를 낮추는 것이 자연스러움.

[Q3-3] State-value vs Action-value 관계

정의:

- $v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$: 상태 s 에서 정책 π 를 따를 때 기대 수익
- $q_{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a]$: 상태 s 에서 행동 a 를 한 뒤 정책 π 를 따를 때 기대 수익

관계식:

...

$$v_{\pi}(s) = \sum_a \pi(a|s) \cdot q_{\pi}(s, a)$$

...

유도: 전체 기댓값의 법칙(law of total expectation)에 의해

...

$$\begin{aligned} v_{\pi}(s) &= E_{\pi}[G_t \mid S_t = s] \\ &= \sum_a \Pr(A_t = a \mid S_t = s) \cdot E_{\pi}[G_t \mid S_t = s, A_t = a] \\ &= \sum_a \pi(a|s) \cdot q_{\pi}(s, a) \end{aligned}$$

...

[Q3-4] Bellman 방정식 유도 (★ 핵심)

****Step 1****: $G_t = R_{t+1} + \gamma \cdot G_{t+1}$

...

$$\begin{aligned} G_t &= R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma \cdot R_{t+3} + \dots) \\ &= R_{t+1} + \gamma \cdot G_{t+1} \end{aligned}$$

...

****Step 2****: 정의에 대입 + 기댓값 선형성

...

$$\begin{aligned} v_\pi(s) &= E_\pi[G_t \mid S_t = s] \\ &= E_\pi[R_{t+1} \mid S_t = s] + \gamma \cdot E_\pi[G_{t+1} \mid S_t = s] \\ &= s \end{aligned}$$

...

****Step 3****: 각 항을 $\sum \pi \cdot p$ 로 전개

$$- E_\pi[R_{t+1} \mid S_t = s] = \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot r(s,a,s')$$

$$- E_\pi[G_{t+1} \mid S_t = s] = \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot v_\pi(s')$$

(다시 $v_\pi(s')$ 의 정의를 사용)

****최종****

...

$$\boxed{v_\pi(s) = \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot [r(s,a,s') + \gamma \cdot v_\pi(s')]} \quad \boxed{\quad}$$

\\

[Q3-5] 1x5 Grid World transition diagram

****상태:**** {L1, L2, L3, L4, L5}, **행동:** {right, left}

(예시 배치) L1=빈칸, L2=폭탄(-2), L3=빈칸, L4=사과(+1), L5=빈칸

****Transition table**** (deterministic, 벽이면 제자리):

s a s' r(s,a,s')
--- --- ---- -----
L1 left L1 0 (벽)
L1 right L2 -2 (폭탄)
L2 left L1 0
L2 right L3 0
L3 left L2 -2
L3 right L4 +1 (사과)
L4 left L3 0
L4 right L5 0
L5 left L4 +1
L5 right L5 0 (벽)

> 시험에서는 그림이 다를 수 있으니 r 값은 그림의 사과/폭탄 위치를 보고 결정.

[Q3-6] 1x2 Grid World, 결정적 정책 ($\gamma = 0.9$)

**** $\mu(L1) = \text{right}, \mu(L2) = \text{right}$ **** (L1→L2: r = +1, L2→L2: r = -1 가정)

****Bellman 방정식:****

\\

$$v(L1) = 1 + 0.9 \cdot v(L2)$$

$$v(L2) = -1 + 0.9 \cdot v(L2)$$

$$\dots$$

****풀이:****

$\dots$

$$v(L2) \cdot (1 - 0.9) = -1 \implies v(L2) = -10$$

$$v(L1) = 1 + 0.9 \cdot (-10) = 1 - 9 = -8$$

$$\dots$$

****답: $v(L1) = -8, v(L2) = -10$ ****

(다른 정책 μ_2, μ_3, μ_4 에 대해서도 같은 방식으로 풀이 가능.)

$---$

**[Q3-7] 4-Grid World 균등 정책 ($L1 = -1, L4 = +1, \gamma = 0.9$)**

****Setup**:** $L1, L4$ 가 종결상태 가정 시 $v(L1) = -1, v(L4) = +1$ 로 고정.

****중간 상태 $L2, L3$ 에 대해 균등 정책**** $\pi(\text{left}|s) = \pi(\text{right}|s) = 0.5$:

$\dots$

$$v(L2) = 0.5 \cdot [0 + 0.9 \cdot v(L1)] + 0.5 \cdot [0 + 0.9 \cdot v(L3)]$$

$$v(L3) = 0.5 \cdot [0 + 0.9 \cdot v(L2)] + 0.5 \cdot [0 + 0.9 \cdot v(L4)]$$

$$\dots$$

대입:

$\dots$

$$v(L2) = 0.5 \cdot 0.9 \cdot (-1) + 0.45 \cdot v(L3)$$

$$= -0.45 + 0.45 \cdot v(L3)$$

$$v(L3) = 0.45 \cdot v(L2) + 0.5 \cdot 0.9 \cdot (1)$$

$$\dots = 0.45 \cdot v(L2) + 0.45$$

연립:

...

$$\begin{aligned} v(L3) &= 0.45 \cdot (-0.45 + 0.45 \cdot v(L3)) + 0.45 \\ &= -0.2025 + 0.2025 \cdot v(L3) + 0.45 \end{aligned}$$

$$\begin{aligned} 0.7975 \cdot v(L3) &= 0.2475 \\ v(L3) &\approx 0.310 \end{aligned}$$

$$\begin{aligned} v(L2) &= -0.45 + 0.45 \cdot 0.310 \approx -0.310 \\ \dots \end{aligned}$$

****답:**** $v(L2) \approx -0.310$, $v(L3) \approx 0.310$

(시험에서 종결 상태 처리 방식에 따라 값 다를 수 있으니 문제의 정의를 확인.)

****[Q3-8]**** 4-Grid World Bellman optimality equation

****중간 상태에 대해:****

...

$$\begin{aligned} v^*(L2) &= \max\{ 0 + 0.9 \cdot v^*(L1), 0 + 0.9 \cdot v^*(L3) \} \\ v^*(L3) &= \max\{ 0 + 0.9 \cdot v^*(L2), 0 + 0.9 \cdot v^*(L4) \} \\ \dots \end{aligned}$$

종결 상태 가정 시: $v^*(L1) = -1$, $v^*(L4) = +1$.

직관: L2에서는 L3쪽으로(positive 방향), L3에서는 L4쪽으로 가는 게 최적.

...

$$v^*(L3) = 0.9 \times 1 = 0.9$$

$v^*(L2) = 0.9 \times 0.9 = 0.81$

...

****Optimal policy****: $\mu^*(L2) = \text{right}$, $\mu^*(L3) = \text{right}$.

📖 Ch.04 Dynamic Programming ★★★

🔍 핵심 개념 상세

1) DP의 본질

****Dynamic Programming****: 복잡한 문제를 작은 부분 문제로 분해하고, 부분 문제의 결과를 ****저장****하여 재사용하는 기법.

- ****장점****: Bellman 방정식(연립방정식)을 직접 풀지 않고 반복적으로 근사

- ****단점****:

- ****차원의 저주****: 상태/행동 공간이 커지면 계산량 폭증
- ****모델 필요****: $p(s'|s,a)$, $r(s,a,s')$ 를 알아야 함 (model-based)

2) Iterative Bellman Equation

****핵심 아이디어****: 추정치 $V_k(s)$ 를 만들고, Bellman 방정식의 우변에 대입하여 새 추정치 $V_{k+1}(s)$ 를 계산. 반복하면 $v_\pi(s)$ 로 수렴.

...

$$V_{k+1}(s) = \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot [r(s,a,s') + \gamma \cdot V_k(s')]]$$

...

이를 ****bootstrapping****이라 부름: "다음 상태의 추정치로 현재 상태의 추정치를 갱신".

****수렴 보장****: $\gamma < 1$ 인 contraction mapping이므로, 어떤 V_0 에서 시작해도 v_π 로 수렴 (Banach fixed-point theorem).

```

> 💡 **쉽게 이해하기 (반복법)**
>
> - **읽는 법:** "지금 있는 추정치  $V_k$ 로 우변을 계산해서 **새 추정치  $V_{k+1}$ ** 을 만든다. 이걸 반복하면 결국 진짜  $v_\pi$ 에 수렴."
> - **첨자 의미:**
>   -  $V_k(s)$ : **k번째 반복**에서의 상태  $s$  추정치 (시간  $t$ 와 무관)
>   -  $k \rightarrow k+1$ : 한 번의 반복 단계
> - **왜 풀리나:** 원래 Bellman은  $|S|$ 개의 미지수를 가진 연립방정식. 직접 행렬로 풀려면 어렵지만, **양변을 좌→우로 한 번씩 갱신**하기만 해도 점점 정답에 가까워짐.
> - **Bootstrapping이란:** **자기 추정값을 다음 추정의 재료로 쓰는 것**.  $V_{k+1}(s)$ 을 만들 때  $V_k(s')$ 를 사용 = 추정으로 추정을 만든다.
> - **비유:** 시험 점수를 모를 때, "옆 친구 점수 추측치"로 내 점수를 추정하고, 그 추정으로 또 옆 친구 점수를 다시 추정하면서 같이 다듬어 가는 것.

```

3) Policy Evaluation 알고리즘

...

입력: 정책 π , threshold θ

초기화: $V(s) = 0$ for all $s \in S^+$ ($S^+ = S \cup \{\text{terminal}\}$)

반복:

$\Delta \leftarrow 0$

for each $s \in S$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot [r(s,a,s') + \gamma \cdot V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

종료 조건: $\Delta < \theta$

출력: $V \approx v_\pi$

...

4) Policy Improvement Theorem

****Statement****: 정책 π 와 π' (deterministic, $\mu'(s)$ 에 의해 정의)에 대해, 모든 s 에서

...

$q_{\pi}(s, \mu'(s)) \geq v_{\pi}(s)$
...

이면, 모든 s 에서

...

$v_{\pi'}(s) \geq v_{\pi}(s)$
...

즉, "한 step만 다르게 행동해도 더 좋다"면 "전체 정책을 바꿔도 더 좋다".

****Greedy improvement****:

...

$\mu'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$
 $= \operatorname{argmax}_a \sum_{s'} p(s'|s, a) \cdot [r(s, a, s') + \gamma \cdot v_{\pi}(s')]$
...

> 💡 ****쉽게 이해하기 (정책 개선 정리)****

>

> - ****읽는 법****

> - 조건: "***모든 자리 s **에서, π 대신 한 번 $\mu'(s)$ 를 따른 뒤 다시 π 로 돌아가도 $v_{\pi}(s)$ 보다 같거나 좋다"

> - 결론: "***그럼 매번 μ' 를 쓰는 새 정책 π' ** 이 모든 자리에서 옛 π 보다 같거나 좋다."

> - ****왜 중요한가****: "한 스텝만 더 좋게 바뀌도 ****전체가 더 좋아진다****"는 보장 → 단계적 개선이 항상 안전하다.

> - ****`argmax\_a`의 뜻****: "가장 큰 값을 만드는 a 를 골라라."

> - ****`max\_a f(a)`**: 가장 큰 ****값****을 반환 (예: 5)

> - ****`argmax\_a f(a)`**: 그 값을 만드는 ****행동****을 반환 (예: 'right')

```
> - **greedy improvement의 의미:** "현재 q값이 가장 큰 행동을 항상 고르는 정책으로 갈아탄다." → 단조 개선(monotonic improvement).
> - **수렴:** 더 이상 개선이 없으면 **최적 정책  $\pi^*$ **. (Bellman optimality 만족)
```

5) Policy Iteration vs Value Iteration

Policy Iteration (정책 반복법)

```
...
```

초기화: 임의의 π

반복:

1. Policy Evaluation: V 를 수렴할 때까지 반복 갱신 (v_π 계산)
 2. Policy Improvement: $\pi' = \text{greedy}(V)$
 3. $\pi == \pi'$ 이면 종료, 아니면 $\pi \leftarrow \pi'$
- ```
...
```

#### #### Value Iteration (가치 반복법)

```
...
```

초기화:  $V(s) = 0$

반복:

```
for each s:
 $V(s) \leftarrow \max_a \sum_{s'} p(s'|s,a) \cdot [r(s,a,s') + \gamma \cdot V(s')]$
종료: $\Delta V < \text{threshold}$
출력: $\pi^* = \text{greedy}(V)$
...
```

#### #### 비교표

| 항목    | Policy Iteration | Value Iteration |
|-------|------------------|-----------------|
| 평가 방식 | 수렴까지 반복          | 1 step만         |
| 개선 방식 | greedy           | (max 통합되어 있음)   |
| 정책 필요 | 필요 (평가 단계에서)     | 불필요 (max로 대체)   |

| 한 iteration 비용 | 큼 (평가 수렴) | 작음 |  
 | 수렴까지 iteration | 적음 | 많음 |  
 | Bellman 식 |  $v_\pi$  Bellman equation | Bellman optimality equation |

---

## ⇒ 예상 문제 정답 풀이

### \*\*[Q4-1]\*\* Iterative Bellman + Bootstrapping

**\*\*Bellman 직접 풀이의 어려움:\*\***

- $|S|$ 개 미지수의 1차 연립방정식  $\rightarrow |S|^3$  행렬 연산 필요
- 환경 모델 정확히 알아야 함
- 상태 공간 크면 비현실적

**\*\*Iterative Bellman 아이디어:\*\***

...

$$V_{\{k+1\}}(s) = \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot [r + \gamma \cdot V_k(s')] ]$$

...

- $V_0$ 에서 시작해 반복 적용  $\rightarrow V_k \rightarrow v_\pi$
- 행렬 역연산 없이 단순 곱·합 반복  $\rightarrow$  큰 문제에도 적용 가능

**\*\*Bootstrapping\*\***: "다음 상태의 추정치  $V_k(s')$ 로 현재 상태의 추정치  $V_{\{k+1\}}(s)$ 를 갱신". 즉 자기 자신의 추정치로 자기 자신을 개선하는 방식.

---

### \*\*[Q4-2]\*\* Iterative Policy Evaluation 의사코드

...

Input: policy  $\pi$ , threshold  $\theta > 0$

Initialize  $V(s) = 0$  for all  $s \in S^+$  ( $V(\text{terminal}) = 0$ )

```

Repeat:
 $\Delta \leftarrow 0$
 For each $s \in S$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{a,s'} \pi(a|s) \cdot p(s'|s,a) \cdot [r(s,a,s') + \gamma \cdot V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
Until $\Delta < \theta$

```

Output:  $V \approx v_\pi$   
 $\dots$

**\*\*구현 팁:\*\*** in-place update (한 배열 사용)가 별도 배열 사용보다 일반적으로 더 빨리 수렴.

---

### ### **\*\*[Q4-3]\*\*** Policy Iteration vs Value Iteration

(위 비교표 참고)

**\*\*핵심 차이:\*\***

1. **\*\*평가 깊이\*\***: PI는 수렴까지, VI는 1 step만.
2. **\*\*정책 명시성\*\***: PI는  $\pi$ 를 명시적으로 유지, VI는  $V$ 만 유지하고 마지막에 greedy로  $\pi$  추출.
3. **\*\*사용 식\*\***: PI는 Bellman equation, VI는 Bellman **\*\*optimality\*\*** equation.
4. **\*\*수렴 속도\*\***: 일반적으로 VI가 단순하고 빠름.

---

### ### **\*\*[Q4-4]\*\*** Policy Improvement Theorem

**\*\*Statement:\*\***

- > 두 결정적 정책  $\pi, \pi'$ 에 대해 모든  $s \in S$ 에서
- >  $q_\pi(s, \mu'(s)) \geq v_\pi(s)$  이면, 모든  $s$ 에서  $v_{\pi'}(s) \geq v_\pi(s)$ .

**\*\*보장하는 바\*\***: greedy improvement ( $\mu'(s) = \operatorname{argmax}_a q_\pi(s,$



a))는 항상 정책을 개선하거나 동일하게 유지함. 즉, **\*\*monotonic improvement\*\*** 보장.

**\*\*Optimal에 도달\*\***: 더 이상 개선이 없으면( $\pi' = \pi$ ), Bellman optimality equation을 만족  $\rightarrow \pi = \pi^*$ .

---

### **\*\*[Q4-5]\*\*** Grid World 손계산 예시

**\*\* (b) Value Iteration 1-step (2-Grid World,  $\gamma = 0.9$ ,  $V_0 = 0$ ):\*\***

...

$$\begin{aligned} V_1(L1) &= \max\{ r(L1, \text{left}, L1) + 0.9 \cdot V_0(L1), \\ &\quad r(L1, \text{right}, L2) + 0.9 \cdot V_0(L2) \} \\ &= \max\{ -1 + 0, \quad 1 + 0 \} \\ &= 1 \end{aligned}$$

$$\begin{aligned} V_1(L2) &= \max\{ 0 + 0, \quad -1 + 0 \} \\ &= 0 \end{aligned}$$

...

(보상 정의:  $L1 \rightarrow L1$  left = -1,  $L1 \rightarrow L2$  right = +1,  $L2 \rightarrow L1$  left = 0,  $L2 \rightarrow L2$  right = -1 가정)

**\*\*다음 step:\*\***

...

$$V_2(L1) = \max\{ -1 + 0.9 \cdot (1), \quad 1 + 0.9 \cdot (0) \} = \max\{ -0.1, 1 \} = 1$$

$$V_2(L2) = \max\{ 0 + 0.9 \cdot (1), \quad -1 + 0.9 \cdot (0) \} = \max\{ 0.9, -1 \} = 0.9$$

...

수렴 시:  $v^*(L2) = 0.9 \cdot v^*(L1)$ ,  $v^*(L1) = 1 + 0.9 \cdot v^*(L2)$   
 $\rightarrow v^*(L1) \approx 5.26$ ,  $v^*(L2) \approx 4.74$ .

---

### ### \*\*[Q4-6]\*\* DP의 한계

1. **\*\*차원의 저주(Curse of Dimensionality)\*\***: 상태 수  $|S|$ 가 커질 때 계산량과 메모리가  $|S|$ 에 비례하여 폭증. 예: 바둑( $|S| \approx 10^{170}$ ) 같은 문제에 직접 적용 불가.
2. **\*\*환경 모델 필요(Model-based)\*\***:  $p(s'|s,a)$ ,  $r(s,a,s')$ 를 미리 알아야 함. 현실 문제에서 이 모델을 정확히 아는 경우는 거의 없음.  
→ MC, TD 같은 model-free 방법이 필요한 이유.

---

## # 📖 Ch.05 Monte Carlo Method ★★

### ## 🔍 핵심 개념 상세

#### ### 1) Monte Carlo의 정의

**\*\*Monte Carlo Method\*\***: 무작위 샘플링을 반복하여 수치적으로 결과를 추정하는 알고리즘.

**\*\*예 1)  $\pi$  계산\*\***: 단위 정사각형에 무작위 점  $N$ 개 → 사분원 안의 점 비율  $\times 4 \rightarrow \pi$  추정.

**\*\*예 2) 주사위 두 개의 합 기댓값\*\***:

- 분포모델:  $P(X = k)$ 를 직접 계산 → 7.0
- 샘플모델: 주사위를  $N$ 번 던져 평균 → 7.0에 근사

#### ### 2) 분포모델 vs 샘플모델

- **\*\*분포모델 (Distribution model)\*\***: 확률 분포를 명시적으로 알고 있음 → 기댓값을 해석적으로 계산.
- **\*\*샘플모델 (Sample model)\*\***: 샘플을 생성할 수 있을 뿐, 분포는 모름 → 평균으로 기댓값 추정.

→ 강화학습에서 환경 모델을 모를 때 샘플모델을 쓸 수 있다는 점에서 MC가 유용.

### ### 3) MC Prediction (Policy Evaluation)

**\*\*아이디어\*\***:  $v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s] \rightarrow$  정책  $\pi$ 를 따라 에피소드를 여러 번 진행하고,  $s$  방문 후의  $G$  평균.

**\*\*알고리즘\*\***

...

Input: policy  $\pi$

Initialize:  $V(s) = 0$ , Returns( $s$ ) = empty list

For each episode:

    Generate episode ( $S_0, A_0, R_1, \dots, S_T$ ) following  $\pi$

$G \leftarrow 0$

    For  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma \cdot G + R_{t+1}$

        Append  $G$  to Returns( $S_t$ )

$V(S_t) \leftarrow \text{average}(\text{Returns}(S_t))$

...

> 💡 **\*\*쉽게 이해하기 (MC prediction 알고리즘)\*\***

>

> - **\*\*핵심 아이디어\*\***: " $v_{\pi}(s)$ 는  $G_t$ 의 평균이니까, \*\*에피소드를 끝까지 돌려보고 실제  $G$  값을 평균\*\*내자."

> - **\*\* $G \leftarrow \gamma \cdot G + R_{t+1}$ 의 뜻\*\***

>   - 에피소드 끝( $t = T-1$ )부터 거꾸로 돌아오며  $G$ 를 누적.

>   - 한 칸 거슬러 갈 때마다 \*\*이전  $G$ 에  $\gamma$ 를 곱하고 새 보상을 더함\*

\*  $\rightarrow$  ' $G_t = R_{t+1} + \gamma \cdot G_{t+1}$ '의 재귀 관계.

>   - **\*\*숫자 예 (보상 [1, 2, 3],  $\gamma=0.9$ )\*\***

>       - 시작:  $G=0$

>       -  $t=2$ :  $G = 0.9 \cdot 0 + 3 = 3$

>       -  $t=1$ :  $G = 0.9 \cdot 3 + 2 = 4.7$

>       -  $t=0$ :  $G = 0.9 \cdot 4.7 + 1 = 5.23$

> - **\*\*Returns( $S_t$ )의 역할\*\***: 그 상태를 방문할 때 얻은  $G$ 들을 \*\*리스트에 쌓아 두고\*\* 평균.

> - **\*\*DP와의 차이\*\***: DP는 \*\*모델로 계산\*\*, MC는 \*\*실제 경험으로 평균\*\*.

**\*\*Incremental 형태:\*\***

...

$$V_n(s) = V_{n-1}(s) + (1/n) \cdot [ G^{(n)} - V_{n-1}(s) ]$$

...

> 💡 **\*\*쉽게 이해하기 (MC incremental)\*\***

>

> - **\*\*읽는 법:\*\*** "n번째 에피소드의 G값으로 V를 한 칸 갱신."

> - **\*\*앞 Ch.02의 점진식과 같은 구조:\*\*** `R\_n` 자리에 `G^(n)` (n번째 episode의 return)이 들어간 것뿐.

> - **\*\*`G^(n)`의 의미:\*\*** "이 상태를 n번째로 방문했던 그 에피소드에서 얻은 누적 보상".

> - **\*\*장점:\*\*** Returns 리스트를 다 안 들고 다녀도 됨. 메모리 일정.

#### ### 4) MC vs DP

- **\*\*DP\*\***: bootstrap ( $V_{k+1}(s)$ 는  $V_k(s')$ 에 의존), 모델 필요.

- **\*\*MC\*\***: 실제 episode의 G를 그대로 사용 (no bootstrap), 모델 불필요. **\*\*Learning\*\*** (실제 경험으로 학습) vs **\*\*Computing\*\*** (DP는 모델로 계산).

#### ### 5) MC Control (Policy Improvement)

**\*\*Q를 추정해야 하는 이유\*\***: V만 알면 정책 개선 시

...

$$\mu(s) = \operatorname{argmax}_a \sum_{s'} p(s'|s,a) \cdot [ r + \gamma \cdot V(s') ]$$

...

처럼 모델 p가 필요함. 반면 Q를 직접 추정하면

...

$$\mu(s) = \operatorname{argmax}_a Q(s, a)$$

...

로 모델 없이 개선 가능.

**\*\*ε-greedy 사용 이유\*\***: greedy만 쓰면 한 번도 안 가본 (s, a) 쌍을 영원히 안 가서 Q 추정 불가능 → exploration 보장 필요.

### ### 6) Exponential Moving Average

1/n 대신 고정  $\alpha$  사용:

...

$$Q_n(s, a) = Q_{n-1}(s, a) + \alpha \cdot [G^n - Q_{n-1}(s, a)]$$

...

전개하면  $G^n$ 의 가중치가  $\alpha \cdot (1-\alpha)^{n-i}$  → 최신 데이터에 더 큰 가중치. 비정상 환경 또는 학습 초기 부정확한 Q를 빠르게 잊고 갱신할 때 유리.

---

## ➡ 예상 문제 정답 풀이

### ### \*\*[Q5-1]\*\* Monte Carlo 개념

**\*\*개념\*\***: 무작위 샘플링을 반복해 분포의 통계량을 추정하는 기법.

**\*\*예시 ( $\pi$  계산)\*\***:

- 단위 정사각형  $[0,1]^2$ 에 N개 점을 균등 분포로 생성
- 원점에서 거리  $\leq 1$ 인 점의 수 M
- $\pi \approx 4 \cdot M / N$  (사분원 면적 비율  $\times 4$ )
- N이 커질수록 정확.

이는 "확률적 시뮬레이션으로 결정론적 양을 추정"하는 MC의 본질을 보여줍니다.

---

### ### \*\*[Q5-2]\*\* DP vs MC 비교 (★)

| 항목 | DP | MC |
|----|----|----|
|----|----|----|

```
|-----|-----|-----|
| 환경 모델 | **필요** (p, r 알아야 함) | **불필요** (샘플만 있으면 됨) |
| 차원의 저주 | 있음 | 비교적 자유로움(샘플링 기반) |
| Episode 종류 | 일회성/지속성 모두 | **일회성만** (지속성 X) |
| Markov property | 필요 | **불필요** (샘플 평균이라) |
| 학습 방식 | Bootstrapping (computing) | Sampling (learning) |
| 평가 시점 | 매 step | 에피소드 종료 후 |
```

---

### \*\*[Q5-3]\*\* Incremental 갱신 유도

...

$$\begin{aligned}
 V_n &= (1/n) \cdot \sum_{i=1..n} G^{(i)} \\
 &= (1/n) \cdot [ G^{(n)} + \sum_{i=1..n-1} G^{(i)} ] \\
 &= (1/n) \cdot [ G^{(n)} + (n-1) \cdot V_{n-1} ] \\
 &= G^{(n)} / n + V_{n-1} - V_{n-1} / n \\
 &= V_{n-1} + (1/n) \cdot ( G^{(n)} - V_{n-1} )
 \end{aligned}$$

...

...

$$\boxed{V_n(s) = V_{n-1}(s) + (1/n) \cdot [ G^{(n)} - V_{n-1}(s) ]}$$

...

---

### \*\*[Q5-4]\*\* Q를 추정해야 하는 이유

V를 사용하면 정책 개선 시

...

$$\mu(s) = \operatorname{argmax}_a \sum_{s'} p(s'|s, a) \cdot [ r + \gamma \cdot V(s') ]$$

...

이 필요하므로 **\*\*환경 모델 p, r을 알아야 함\*\***.

Q를 직접 추정하면

...

$$\mu(s) = \operatorname{argmax}_a Q(s, a)$$

...

로 모델 없이 개선 가능 → MC, TD 같은 model-free 방법이 V 대신 Q를 사용하는 이유.

---

### **\*\*[Q5-5]\*\*** EMA의 장점과 가중치 변화

**\*\*식\*\***:  $Q_n = Q_{n-1} + \alpha \cdot (G^{(n)} - Q_{n-1})$

**\*\*가중치 전개\*\***:

...

$$\begin{aligned} Q_n &= (1 - \alpha) \cdot Q_{n-1} + \alpha \cdot G^{(n)} \\ &= (1 - \alpha)^n \cdot Q_0 + \sum_{i=1..n} \alpha \cdot (1 - \alpha)^{n-i} \cdot G^{(i)} \end{aligned}$$

...

가중치  $\alpha \cdot (1 - \alpha)^{n-i}$ 는 i가 작을수록(과거) 지수적으로 감소 → 최신 데이터에 큰 가중치.

**\*\*장점\*\***

1. 비정상 환경에서 변화 추종 가능
2. 초기의 부정확한 Q를 빠르게 잊음
3. 메모리 일정 (n 무관)

**\*\*단점\*\*** stationary 환경에서 표본평균보다 분산 큼.

---

### **\*\*[Q5-6]\*\***  $\epsilon, \alpha$  변화의 영향

- **$\epsilon \uparrow$** : 탐색  $\uparrow \rightarrow$  다양한 (s, a) 경험  $\rightarrow$  학습 안정적이지만 보상은 낮을 수 있음
- **$\epsilon \downarrow$** : 활용  $\uparrow \rightarrow$  빠른 수렴이지만 local optimum 위험
- **$\alpha \uparrow$** : 최신 데이터 가중  $\uparrow \rightarrow$  빠른 갱신이지만 분산 큼
- **$\alpha \downarrow$** : 과거 평균에 가까움  $\rightarrow$  안정적이지만 변화 느림

5x5 Grid World에서는 보통  $\epsilon \approx 0.1$ ,  $\alpha \approx 0.1$  정도가 균형점.

---

## # 📖 Ch.06 Temporal Difference Method ★★★

### ## 🔍 핵심 개념 상세

#### ### 1) TD = MC + DP

| 방법        | Bootstrapping                    | Sampling                    |
|-----------|----------------------------------|-----------------------------|
| DP        | 0 ( $V_k(s')$ 로 $V_{k+1}(s)$ 계산) | X (모델로 모든 $s'$ 고려)          |
| MC        | X (실제 G 사용)                      | 0 (샘플 episode)              |
| <b>TD</b> | <b>0</b> ( $V(S_{t+1})$ 사용)      | <b>0</b> (실제 transition 샘플) |

$\rightarrow$  TD는 두 방법의 장점을 결합:

- 모델 불필요 (MC처럼)
- 매 step 갱신 가능, 지속성 과제 가능 (DP처럼)
- 분산 작음 (MC보다)

#### ### 2) TD(0) 갱신식

**유도**: Bellman 방정식

...

$$v_{\pi}(s) = E[ R_{t+1} + \gamma \cdot v_{\pi}(S_{t+1}) \mid S_t = s ]$$

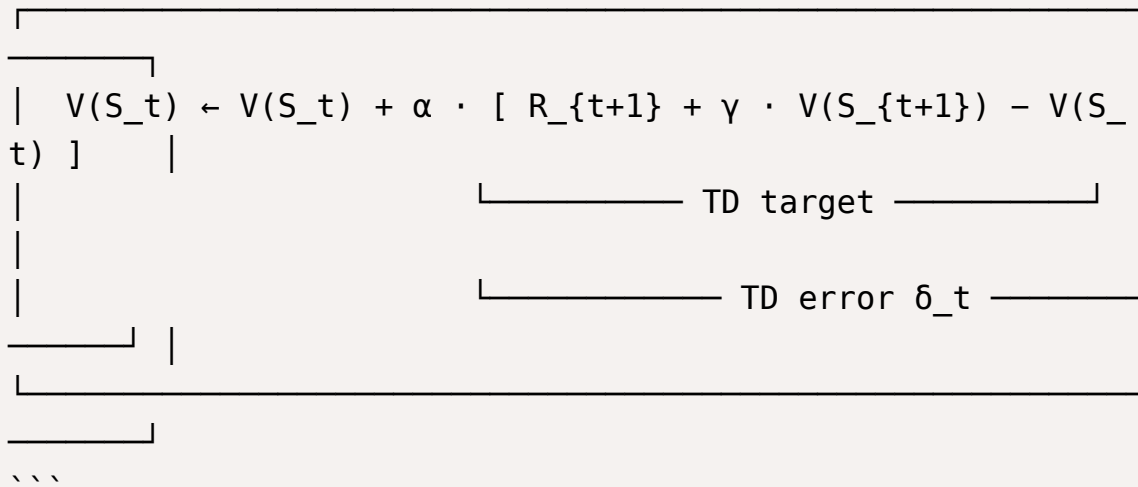
...



이 기댓값을 한 샘플로 근사:  $R_{t+1} + \gamma \cdot V(S_{t+1})$  (TD target  $t$ ).

갱신:

...



**\*\*TD error:\*\***

...

$$\delta_t = R_{t+1} + \gamma \cdot V(S_{t+1}) - V(S_t)$$

...

> 💡 **\*\*쉽게 이해하기 (TD(0))\*\***

>

> - **\*\*읽는 법:\*\*** "한 스텝만 가보고 ( $R_{t+1}$ ,  $S_{t+1}$ ) 얻으면, \*  
\*그 정보로 즉시  $V$ 를 한 번 고친다\*\*."

> - **\*\*각 부분 의미:\*\***

> - **\*\* $R_{t+1} + \gamma \cdot V(S_{t+1})$ \*\*** = **\*\*TD target\*\***: "이번  
에 받은 보상 + 다음 상태의 추정 가치(할인됨)" = 새 추정 후보.

> - **\*\* $V(S_t)$ \*\*** = **\*\*옛 추정\*\***: 지금까지 알고 있던 그 자리의  
가치.

> - **\*\* $\delta_t = \text{target} - \text{옛 추정}$ \*\*** = **\*\*TD error\*\***: 새 정보가  
옛 추정과 얼마나 다른지.

> - **\*\* $\alpha \cdot \delta_t$ \*\*** = 차이의 일부만큼 옛 추정을 새 쪽으로 끌어당  
김.

> - **\*\*MC와의 차이:\*\***

> - MC:  $G_t$  (에피소드 끝까지 진짜 누적 보상) 사용 → 정확하지만 분산 큼, 끝나야만 갱신 가능.

> - TD:  $R_{t+1} + \gamma \cdot V(S_{t+1})$  (한 스텝 + 추정) 사용 → \*\*편향 있지만 매 스텝 갱신\*\*, 분산 작음.

> - \*\*Bootstrap의 의미:\*\* TD target에 \*\*자기 추정  $V(S_{t+1})$  \* 이 들어 있어서, 추정으로 추정을 갱신 (= bootstrap).

> - \*\*숫자 예 ( $V(S_t)=2, R=1, \gamma=0.9, V(S_{t+1})=3, \alpha=0.1$ ):\*\*

> -  $target = 1 + 0.9 \cdot 3 = 3.7$

> -  $\delta = 3.7 - 2 = 1.7$

> - 새  $V(S_t) = 2 + 0.1 \cdot 1.7 = 2.17$

> - "지금까지 2라고 봤는데, 한 스텝 가보니 3.7쯤 같다 → 그 차이의 10%만큼 올림."

### ### 3) TD vs MC 비교

| 항목     | MC           | TD                                            |
|--------|--------------|-----------------------------------------------|
| 갱신 시점  | 에피소드 종료 후    | 매 step (1-step)                               |
| 사용 정보  | 실제 G (전체 미래) | $R + \gamma \cdot V(S')$ (1-step + bootstrap) |
| 분산     | 큼 (G가 긴 합)   | 작음 (1-step만)                                  |
| 편향     | 작음 (실제 값)    | 있음 (V 추정치 사용)                                 |
| 지속성 과제 | X            | 0                                             |

### ### 4) SARSA (On-policy TD Control)

**\*\*5튜플\*\*** ( $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$ )로 갱신:

...

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R_{t+1} + \gamma \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

...

> 💡 **\*\*쉽게 이해하기 (SARSA)\*\***

>

> - **\*\*이름의 어원:\*\*** **\*\*S\*\***tate, **\*\*A\*\***ction, **\*\*R\*\***eward, **\*\*S\*\***tate', **\*\*A\*\***ction' - 갱신에 쓰는 5요소의 머리글자.

> - **\*\*TD(0)과 비교:\*\***  $V(s)$ 가  $Q(s, a)$ 로 바뀐 것 + 다음 상태의 가

치를 **실제로 고른 행동  $A_{t+1}$ 의 q값**으로 사용.

> - TD(0):  $R + \gamma \cdot V(S')$

> - SARSA:  $R + \gamma \cdot Q(S', A')$  ←  $A'$ 은 실제 정책으로 선택한 행동

> - **On-policy의 의미:** **"행동을 고른 정책(예:  $\epsilon$ -greedy)과 평가/개선하는 정책이 같음"** → 자기 경험으로 자기를 개선.

> - **흐름:**

> 1. 상태  $S$ 에서  $\epsilon$ -greedy로  $A$ 를 골라 실행

> 2. 보상  $R$ , 다음 상태  $S'$  관측

> 3. **다음 상태에서  $A'$ 을 미리  $\epsilon$ -greedy로 결정**

> 4.  $Q(S, A)$ 를 위 식으로 갱신

> 5.  $S \leftarrow S', A \leftarrow A'$  후 반복

**알고리즘:**

...

Initialize  $Q(s, a)$  arbitrarily

For each episode:

$S \leftarrow$  initial state

$A \leftarrow \epsilon$ -greedy( $Q, S$ )

    Repeat (for each step):

        Take  $A$ , observe  $R, S'$

$A' \leftarrow \epsilon$ -greedy( $Q, S'$ )

$Q(S, A) \leftarrow Q(S, A) + \alpha \cdot [R + \gamma \cdot Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A'$

    until  $S$  is terminal

...

**On-policy:** 행동을 선택한 정책( $\epsilon$ -greedy)과 평가/개선하는 정책이 동일.

### 5) On-policy vs Off-policy

- **On-policy:** 행동 정책(behavior policy) = 대상 정책(target policy)

- 예: SARSA, MC Control

- "내가 직접 한 경험으로 내 정책 개선"
- **\*\*Off-policy\*\***: 행동 정책  $\neq$  대상 정책
  - 예: Q-Learning, Off-policy SARSA
  - "다른 정책의 경험으로 내 정책 개선"
  - 장점: 데이터 재사용, exploration과 exploitation 분리, replay buffer 사용 가능

### ### 6) Importance Sampling (IS)

**\*\*문제\*\***: 분포  $b$ 에서 샘플링한 데이터로 분포  $\pi$ 의 기댓값을 추정.

...

$$\begin{aligned} E_{\pi}[x] &= \sum_x \pi(x) \cdot x \\ &= \sum_x b(x) \cdot \left( \pi(x) / b(x) \right) \cdot x \\ &= E_b[ \rho(x) \cdot x ] \end{aligned}$$

...

**\*\*Importance ratio\*\***:  $\rho(x) = \pi(x) / b(x)$

**\*\*MC 추정\*\***:

...

$$E_{\pi}[x] \approx (1/n) \cdot \sum_{i=1..n} \rho(x^{(i)}) \cdot x^{(i)}, \quad x^{(i)} \sim b$$

...

**\*\*문제점\*\***:  $\pi$ 와  $b$ 가 많이 다르면  $\rho$ 가 매우 커지거나 작아져서 추정의 분산이 폭발.

> 💡 **\*\*쉽게 이해하기 (Importance Sampling)\*\***

>

> - **\*\*상황\*\***: 우리가 알고 싶은 건 **\*\*정책  $\pi$ 에서의 평균\*\***인데, 데이터는 **\*\*다른 정책  $b$ \*\***로 모았다.

> - **\*\*트릭\*\***:  $\pi(x)/b(x) \cdot b(x) = \pi(x)$  이므로,  $b$  분포로 샘플하고 \*  
\*각 샘플에  $\rho = \pi/b$  가중치\*\*를 곱하면  $\pi$  분포의 기댓값과 같아짐.

> - **\*\*3줄 유도\*\***

> - 첫 줄: 원하는 식 ( $\pi$ 에서의 기댓값)

> - 둘째 줄: 분자/분모에  $b(x)$  곱하기 = 같은 식 (수학적으로 동일)

> - 셋째 줄:  $b$  분포의 기댓값으로 다시 적기  $\rightarrow$  **\*\* $b$ 로 샘플하고  $\rho$ 를**

곱한 것의 평균\*\* =  $\pi$ 의 기댓값"

> - \*\*` $\rho(x) = \pi(x)/b(x)$ `의 직관:\*\*

> -  $\pi$ 에서 자주 나오는데  $b$ 에서 잘 안 나오는  $x \rightarrow \rho$ 가 큼  $\rightarrow$  "더 많이 봤다고 치고 싶다"  $\rightarrow$  가중치  $\uparrow$

> - 반대면 가중치  $\downarrow$

> - \*\*분산 폭발 문제:\*\*  $\pi$ 와  $b$ 가 매우 다르면  $\rho$ 가 1000, 0.001 같이 극단적이 됨  $\rightarrow$  평균이 불안정.

> - \*\*숫자 예:\*\*  $\pi(x=A) = 0.9$ ,  $b(x=A) = 0.1 \rightarrow \rho = 9$ .  $b$ 로 한 번  $A$ 가 나오면 9배 가중. 이런 게 자주 일어나면 분산 큼.

### ### 7) Off-policy SARSA

**Target policy**  $\pi$  : greedy

**Behavior policy**  $b$  :  $\epsilon$ -greedy

...

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R + \gamma \cdot \rho \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

...

여기서  $\rho = \pi(A_{t+1} | S_{t+1}) / b(A_{t+1} | S_{t+1})$ .

### ### 8) Q-Learning

**Bellman optimality equation**의 sampling 버전:

...

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

...

> 💡 **\*\*쉽게 이해하기 (Q-Learning)\*\***

>

> - **\*\*SARSA와의 한 글자 차이:\*\***

```

> - SARSA: `... + γ · Q(S', A') - ...` (실제 다음 행동 A')
> - Q-Learning: `... + γ · max_a Q(S', a) - ...` (다음 상태에서 **최선의 행동 가정**)
> - **`max_a Q(S', a)`의 뜻:** "다음 상태 S'에서 **가장 좋은 행동의 q값**".
> - 실제로 그 행동을 하든 안 하든 상관없음 - 그저 **현재 추정 기준 최선**을 가정.
> - **Off-policy의 의미:**
> - **Behavior policy:** ε-greedy로 행동 (탐색 포함)
> - **Target policy:** greedy (max를 함함)
> - 둘이 달라서 off-policy. 하지만 **max는 결정론적**이라 IS 없이도 안전.
> - **Bellman optimality와의 연결:**
> - 원래 식: $q^*(s,a) = \sum p(s'|s,a) \cdot [r + \gamma \cdot \max_{a'} q^*(s', a')]$
> - Q-Learning: 한 샘플 (S, A, R, S')로 그 기댓값을 근사 → $R + \gamma \cdot \max_a Q(S', a)$ 가 target.
> - **장점:**
> - **Off-policy** → 옛 데이터 재활용(replay buffer), 다른 사람의 경험 사용 가능
> - **IS 불필요** → 분산 폭발 없음 → DQN의 기반
> - **단점/주의:**
> - **위험한 경로도 max로 학습** → Cliff walking에서 절벽 옆 최단경로 학습(실제 행동 시 ε-greedy로 떨어질 수도).
> - 보수적으로는 SARSA가 더 안전한 정책을 학습.
> - **숫자 예 (Q(S,A)=2, R=1, γ=0.9, Q(S', a) = [3, 5, 1], α=0.1):**
> - $\max_a Q(S', a) = 5$
> - $\text{target} = 1 + 0.9 \cdot 5 = 5.5$
> - 새 $Q(S,A) = 2 + 0.1 \cdot (5.5 - 2) = 2.35$

```

**\*\*알고리즘:\*\***

...

Initialize Q(s, a)

For each episode:

    S ← initial state

```

Repeat:
 A ← ϵ -greedy(Q, S)
 Take A, observe R, S'
 Q(S, A) ← Q(S, A) + $\alpha \cdot [R + \gamma \cdot \max_a Q(S', a) - Q(S, A)]$
 S ← S'
until S is terminal
...

```

**\*\*Off-policy인데 IS 불필요한 이유\*\***: target에서 다음 행동을 b로부터 샘플링하지 않고 max로 직접 선택하므로, target에 b의 영향이 없음.

### ### 9) SARSA vs Q-Learning

| 항목            | SARSA                              | Q-Learning                         |
|---------------|------------------------------------|------------------------------------|
| 기반 식          | Bellman equation                   | Bellman optimality equation        |
| Target        | $R + \gamma \cdot Q(S', A')$       | $R + \gamma \cdot \max_a Q(S', a)$ |
| 다음 행동         | 실제 샘플링한 A' 사용                      | max로 가상의 최적 행동                     |
| On/Off-policy | On                                 | Off                                |
| IS 필요?        | (on-policy) 불필요                    | 불필요 (max 사용)                       |
| 학습 안정성        | 보수적 (실제 정책 반영)                     | 적극적 (greedy 학습)                    |
| 위험 회피         | 0 (실제 $\epsilon$ -greedy의 위험까지 학습) | X (greedy 가정으로 위험 무시)              |

**\*\*유명한 예시 (Cliff walking)\*\***: SARSA는 절벽에서 멀리 돌아가는 안전 경로를 학습, Q-Learning은 절벽 바로 옆 최단 경로를 학습 (실제로는  $\epsilon$ -greedy로 가끔 떨어짐).

---

## ☞ 예상 문제 정답 풀이

### ### \*\*[Q6-1]\*\* TD vs DP vs MC 비교 (★)

| 항목            | DP | MC | TD |
|---------------|----|----|----|
| Bootstrapping | 0  | X  | 0  |

|          |             |              |               |
|----------|-------------|--------------|---------------|
| Sampling | X           | 0            | 0             |
| 모델 필요    | 0           | X            | X             |
| Episode  | 일회성/지속성     | 일회성만         | 둘 다           |
| 갱신 시점    | 매 step (계산) | episode 종료 후 | 매 step        |
| 분산       | (해당 없음)     | 큼            | 작음            |
| 편향       | (정확)        | 무편향          | 편향 (V 추정치 사용) |

---

### \*\*[Q6-2]\*\* TD(0) 갱신식 (★)

...

$$V(S_t) \leftarrow V(S_t) + \alpha \cdot \left[ \underbrace{R_{t+1} + \gamma \cdot V(S_{t+1})}_{\text{TD target}} - \underbrace{V(S_t)}_{\text{TD error } \delta_t} \right]$$

...

**\*\*MC와의 차이:\*\***

- MC:  $V(S_t) \leftarrow V(S_t) + \alpha \cdot [G_t - V(S_t)]$  ( $G_t$ 는 episode 끝까지 누적)
- TD:  $V(S_t) \leftarrow V(S_t) + \alpha \cdot [R_{t+1} + \gamma \cdot V(S_{t+1}) - V(S_t)]$  (1 step + bootstrap)

→ TD는 1-step만 보고 다음 상태의 추정치  $V(S_{t+1})$ 로 나머지를 대체. 그래서 매 step 갱신 가능 (지속성 과제 가능)이고 분산이 작음.

---

### \*\*[Q6-3]\*\* SARSA 알고리즘

...

Input:  $\epsilon \in (0,1), \alpha \in (0,1], \gamma \in (0,1]$   
Initialize  $Q(s, a)$  arbitrarily,  $Q(\text{terminal}, \cdot) = 0$

For each episode:  
    Initialize  $S$   
    Choose  $A \leftarrow \epsilon\text{-greedy}(Q, S)$



```

 For each step:
 Take action A, observe R, S'
 Choose A' $\leftarrow \epsilon$ -greedy(Q, S')
 $Q(S, A) \leftarrow Q(S, A) + \alpha \cdot [R + \gamma \cdot Q(S', A') - Q(S, A)]$
 S $\leftarrow$ S' ; A $\leftarrow$ A'
 until S is terminal
 ...

```

**\*\*이름의 어원\*\***: 갱신에 사용하는 5튜플 (\*\*S\*\*tate, \*\*A\*\*ction, \*R\*\*eward, \*\*S\*\*tate', \*\*A\*\*ction')의 머리글자.

---

### ### \*\*[Q6-4]\*\* On-policy vs Off-policy (★)

- **\*\*On-policy\*\***: 행동을 결정하는 정책(behavior)과 학습하는 정책(target)이 **\*\*같음\*\***.
  - 예: SARSA –  $\epsilon$ -greedy로 행동하고  $\epsilon$ -greedy를 평가/개선.
  - 예: MC Control ( $\epsilon$ -soft)
- **\*\*Off-policy\*\***: behavior와 target이 **\*\*다름\*\***.
  - 예: Q-Learning –  $\epsilon$ -greedy로 행동하지만 greedy 정책을 학습.
  - 예: Off-policy SARSA – behavior =  $\epsilon$ -greedy, target = greedy
  - 장점: 옛 데이터 재활용 (replay buffer), 다른 사람/시뮬레이터의 데이터 활용 가능, 탐색-활용 분리.

---

### ### \*\*[Q6-5]\*\* Importance Sampling

**\*\*식\*\***:

...

$$E_{\pi}[X] = E_b[ (\pi(X) / b(X)) \cdot X ] = E_b[ \rho(X) \cdot X ]$$

...

**\*\*Off-policy SARSA에서  $\rho$ 가 곱해지는 위치\*\***: TD target의 next-a

ction 부분.

...

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R_{t+1} + \gamma \cdot \rho \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

...

여기서  $\rho = \pi(A_{t+1} | S_{t+1}) / b(A_{t+1} | S_{t+1})$ .

이는 "behavior policy로 샘플링한  $A_{t+1}$ 을 target policy 분포의 기댓값으로 보정"하는 역할.

**\*\*문제점\*\***:  $\pi$ 와  $b$ 가 많이 다르면  $\rho$ 가 매우 크거나 작아져 분산 폭발 → 학습 불안정.

---

### **\*\*[Q6-6]\*\*** SARSA vs Q-Learning 5가지 관점 비교 (★ 핵심)

| 관점                | SARSA                        | Q-Learning                             |
|-------------------|------------------------------|----------------------------------------|
| (1) Bellman 식     | Bellman equation             | Bellman <b>**optimality**</b> equation |
| (2) Target        | $R + \gamma \cdot Q(S', A')$ | $R + \gamma \cdot \max_a Q(S', a)$     |
| (3) On/Off-policy | On-policy                    | Off-policy                             |
| (4) IS 필요         | 불필요 (on-policy)              | 불필요 (max 사용)                           |
| (5) 학습 특성         | 실제 정책 반영, 보수적·안전             | greedy 가정, 적극적·최적                      |

**\*\*Cliff walking 직관\*\***: SARSA는  $\epsilon$ -greedy의 무작위성이 절벽 추락 위험을 학습에 반영하므로 안전 경로 선호. Q-Learning은 max만 보므로 최단 경로 학습하지만 학습 중에는  $\epsilon$ -greedy로 자주 떨어짐.

---

### **\*\*[Q6-7]\*\*** Q-Learning이 IS 불필요한 이유

Q-Learning의 target:

...

$R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a)$

...

여기서 다음 행동을 behavior policy  $b$ 로 **샘플링**하는 게 아니라,  $Q$ 를 통해 **결정론적으로 max** 선택. 즉, target 계산에  $b$ 의 분포가 등장하지 않으므로 분포 보정(IS)이 불필요.

(반면 off-policy SARSA는 target에  $Q(S_{t+1}, A_{t+1})$ 이 있고, 여기서  $A_{t+1} \sim b$ 이므로  $\pi$  분포로 보정 필요.)

→ Q-Learning은 **off-policy의 장점**(데이터 재활용 등)을 누리면서 **IS의 단점**(분산 폭발)을 피하므로 매우 안정적이고 강력. DQN의 기반이 됨.

---

### **[Q6-8]** 5x5 Grid World Q-Learning 결과

학습 후 각 상태의  $V(s) = \max_a Q(s, a)$ 는 보상(+1, -1)의 위치를 향하는 방향으로 그라디언트를 형성하고, 정책  $\pi^*$ 는 각 상태에서 최대  $Q$ 를 가진 화살표( $\uparrow \downarrow \leftrightarrow$ )로 표시됨.

( $\gamma = 0.9$ ,  $\epsilon = 0.1$  정도로 충분한 episode를 학습하면 +1 칸 인접 상태들은  $V \approx 0.9$ , 그 다음은 0.81, ... 같은 식으로 지수 감소.)

---

#  종합 정리

## 1. Policy Evaluation 갱신식 비교 (반드시 암기!)

| 방법 | 식                                                                                    | 모델 | Bootstrap | Sampling |
|----|--------------------------------------------------------------------------------------|----|-----------|----------|
| DP | $V_{k+1}(s) = \sum_{a,s'} \pi(a s) \cdot p(s' s,a) \cdot [r + \gamma \cdot V_k(s')]$ | 0  | 0         | X        |
| MC | $V(s) \leftarrow V(s) + \alpha \cdot [G - V(s)]$                                     | X  | X         | 0        |

```
| **TD** | $V(S_t) \leftarrow V(S_t) + \alpha \cdot [R + \gamma \cdot V(S_{t+1}) - V(S_t)]$ | X | 0 | 0 |
```

## ## 2. Control 알고리즘 분류

```
알고리즘	평가	개선	On/Off
Policy Iteration	DP iterative eval	greedy	(model-based)
Value Iteration	DP 1-step + max	(max로 통합)	(model-based)
MC Control	episode 후 Q	ϵ -greedy	On
SARSA	TD Q	ϵ -greedy	On
Off-policy SARSA	TD + IS	greedy / ϵ -greedy	Off
Q-Learning	TD + max	greedy	Off
```

## ## 3. 단답형 빈출

```
질문	답
강화학습이 푸는 문제 유형	Sequential decision-making
Multi-armed bandit에 정의되지 않는 요소	State
Continuing task에서 $\gamma = 1$ 이면?	G가 발산
Markov property 한 줄 표현	"Future $\perp$ Past $\setminus$ Present"
Bellman equation의 본질	연립방정식 문제
Bellman optimality equation의 본질	최적화 문제
DP의 두 한계	차원의 저주, 모델 필요
MC가 지속성 과제에 부적합한 이유	episode 끝나야 G 계산 가능
TD = ? + ?	MC(sampling) + DP(bootstrapping)
SARSA의 어원	(S, A, R, S', A')
Q-Learning이 기반한 식	Bellman optimality equation
Q-Learning의 발전형	DQN (2013, DeepMind)
```

## ## 4. 시험장 '1분 점검' 표 (한눈에 보기)

...

|                                  |
|----------------------------------|
| MDP 5튜플: (S, A, T, r, $\gamma$ ) |
|----------------------------------|

```

| Markov: $P(S' \mid \text{모든 과거}) = P(S' \mid S)$ |
| Return: $G_t = R_{\{t+1\}} + \gamma \cdot G_{\{t+1\}}$ |
|
| Bellman (state): |
| $v(s) = \sum \pi \cdot p \cdot [r + \gamma \cdot v(s')]$ |
| Bellman (optimal): |
| $v^*(s) = \max_a \sum p \cdot [r + \gamma \cdot v^*(s')]$ |
|
| MC update: |
| $V(s) \leftarrow V(s) + \alpha \cdot (G - V(s))$ |
| TD update: |
| $V(s) \leftarrow V(s) + \alpha \cdot (R + \gamma \cdot V(s') - V(s))$ |
|
| SARSA (on-policy): |
| $Q(S, A) \leftarrow Q + \alpha \cdot [R + \gamma \cdot Q(S', A') - Q]$ |
| Q-Learning (off-policy): |
| $Q(S, A) \leftarrow Q + \alpha \cdot [R + \gamma \cdot \max_a Q(S', a) - Q]$ |
|_____

```

...

---

**\*\*시험 화이팅! 🎯\*\***

**\*\*행운을 빕니다.\*\***